ChatGPT

电子票根 / 生活纪念品数字收藏系统：深度技术调研与架构建议

严密逻辑链：产品定位 → 数据与业务语义 → 离线能力边界 → Android 近场通信 → OCR/3D 处理流水线 → 云端存储与同步 → 网络编程难度证明 → 风险收敛 → 一个月可交付版本。

本轮调研日期为 **2026 年 10 月 13 日**。我把你目前的构想按“真正要做成一个产品”和“必须能向网络编程老师证明技术难度”两个视角重新审了一遍。

先给结论：

> **这个项目值得继续，而且不需要为了冲 1.30 再硬塞 NAT 穿透、WebRTC、区块链等不相关技术。**
>
> 但它应该从“电子票务收集 App”进一步定义成：
>
> **一个把现实生活中的票根、标签、照片和视频转化为结构化数字纪念品，并允许用户在无互联网环境下把这份“记忆卡片”直接递给身边朋友的 Local-first 多模态收藏系统。**
>
> 真正能够支撑高难度的，不是 OCR、Three.js、Meshy、COS 这些技术名词的数量，而是：**离线 P2P 可靠传输 + 自定义应用协议 + Local-first 同步 + 异步多节点计算 + 对象存储 + 多模态内容处理**能够形成完整闭环。

我同时整理了完整文档、技术决策矩阵和三张架构图：

下载完整深度调研文档 DOCX

下载技术决策矩阵 XLSX

下载总体架构图 PNG

下载离线分享协议时序图 PNG

下载技术价值—风险矩阵 PNG

项目定位与创意审视

你现在真正想做的，其实已经不应该再叫传统意义上的“电子票务”。

“票务 Ticketing”容易让人理解成：

> 买票 → 验票 → 核销 → 转赠票权 → 防伪 → 所有权。

而你的核心实际上是：

现实物件 / 一段经历 → 数字化 → 结构化 → 视觉化 → 收藏 → 回顾 → 分享给朋友。

所以更准确的产品对象是 **Digital Memorabilia / Memory Artifact，数字纪念品 / 记忆物件**。

例如：

```
电影票
→ OCR
→ 《沙丘3》
→ 影院、日期、座位
→ 自动匹配电影海报
→ 生成数字票根卡片
→ 2.5D / 3D 展示
→ 收藏

奶茶杯贴纸
→ OCR
→ 品牌、饮品名称
→ 保存原始照片
→ 添加“这个很好喝”
→ 生成纪念卡

探店视频
→ 用户拍摄视频
→ 店名、菜名、推荐语
→ 生成“探店纪念卡”
→ 分享给朋友
```

朋友接收到的也不是“票的所有权”，而是：

一份有结构、有视觉、有背景信息的推荐和纪念品。

这条定义非常关键，因为它直接解决了之前“交换”这个词造成的业务逻辑问题。

A 看过电影，不代表 B 收到 A 的电影卡以后也看过电影。

因此数据层必须至少在逻辑上区分：

对象	代表什么	示例
Subject	客观内容对象	《沙丘3》、某家店、某款饮料
Experience	某个人的真实经历	A 在 10 月 5 日看了《沙丘3》
Collectible Snapshot	某次固定的分享版本	海报 + 票面 + A 的推荐语
Collection Entry	某个人如何收藏它	B 收到以后标记“想看”
Asset	实际媒体文件	JPG、MP4、GLB、纹理

这样你以后统计的时候不会发生：

```
A 看过电影
↓
A 分享给 B
↓
B 的“看过电影数量 +1”
```

这种明显错误。

正确逻辑应该是：

```
A：已看
      |
      | 分享 Snapshot
      ▼
B：朋友推荐 / 想看
      |
      | B 后来真的去看
      ▼
B：新增自己的 Experience
```

这实际上会成为这个项目非常漂亮的一条业务主线。

创意并不是“票根 OCR 首创”

这里必须诚实。

截至 **2026 年 10 月 13 日**，已经有相当接近的产品。

PocketStubs 的官方 Google Play 页面已经明确提供实体电影票根扫描并自动加入观看历史，同时包含影视日志、社交互动和 AI Journey Art；它在 **2026 年 10 月 7 日**刚更新过。 [1]

TicketShelf 则已经进一步扩展到演唱会、体育、展览、电影和旅行，并提供 AI 辅助票根录入、照片与周边收藏、地点地图、年度回顾和朋友共同经历等功能；其页面显示于 **2026 年 9 月 8 日**更新。 [2]

所以这些都不能作为你们的核心创新声明：

OCR 识别票根 ❌
自动生成电子票根 ❌
电影海报匹配 ❌
多种票根收藏 ❌
年度回顾 ❌
AI 艺术图 ❌

但从我本轮检查到的这两类直接竞品官方说明中，**没有看到它们把“无互联网附近设备之间直接递送结构化收藏包”与“2.5D/3D 数字纪念品”组合成产品核心体验**。这只能说明在本轮定向检索中没有看到，而不能证明全球不存在同类产品。 [3]

所以你们应该把差异化落在：

> **Capture → Understand → Beautify → Keep → Hand it to a friend**

中文可以理解为：

> **拍下来 → 看懂它 → 把它变漂亮 → 收藏起来 → 当面递给朋友。**

这比“电子票根管理 App”强得多。

我的主观产品判断是：

维度	评价
单纯票根数字化创新性	中
多模态生活纪念品定位	中高
离线“递卡片”交互	高
2D/2.5D/3D 渐进视觉表达	高
网络编程结合自然程度	高
课程展示效果	很高

尤其是“**递一张收藏卡给身边的人**”，我认为应该成为你们以后 Demo 的标志性动作，而不是藏在设置里的一个文件传输功能。

技术路线与可复用的开源方案

本轮最重要的发现是：**没有必要自己造 OCR、图片转 3D、SQLite 封装、Blender 后台任务运行时等基础轮子。**

本轮定向搜索没有找到一个成熟开源项目，能够直接覆盖“多类型票根 OCR + 数字收藏 + 3D + Local-first + 离线 P2P 分享”这一完整产品，因此现实路线不是 fork 一个现成项目改 UI，而是**用成熟组件拼成自己的业务系统**。

近场通信：Wi-Fi Direct 与 Nearby 应该分层，而不是二选一写死

Google Nearby Connections 本身就是为附近设备离线 P2P 发现、建立连接和交换数据设计的。官方说明它会组合使用 Bluetooth、BLE 和 Wi-Fi，并负责隐藏不同无线传输层的复杂性，同时提供经过加密的连接。它支持 Bytes、File 和 Stream 三种 payload；其中 Bytes 有大小限制，File 可以处理大文件，并有进度、取消和失败通知。 [4]

对于你们这种一对一“递收藏品”的场景，Nearby 的 `P2P_POINT_TO_POINT` 策略尤其匹配；Google 将它定位为一对一、强调最高吞吐量的策略，适合大文件分享。 [5]

但 Nearby 的问题恰好也是课程评分的问题：

> **它帮你做得太多了。**

老师如果问：

“你们的网络编程在哪？”

你回答：

“我们调用 Nearby Connections SDK，然后调用 `sendPayload()`。”

那技术深度展示会明显下降。

而 Android Wi-Fi Direct 官方 API 可以让支持 Wi-Fi P2P 的设备在没有传统 Access Point 的情况下直接连接；应用负责设备发现、连接、建立 P2P group，并可以在连接建立后进行自己的 Socket 通信。Android 13 以后还涉及 `NEARBY_WIFI_DEVICES` 等权限以及设备能力检查。 [6]

这正适合网络编程课程。

因此我现在推荐的不是：

```
Nearby OR Wi-Fi Direct
```

而是：

```
       Memorabilia Transfer Protocol
                  MTP
                   |
              Transport API
             /           \
            /             \
Wi-Fi Direct + TCP      Nearby
   课程实现              兼容/备用
```

最优课程路线：先实现 Wi-Fi Direct + TCP。

如果实机测试发现不同品牌 Android 之间 Wi-Fi Direct 的发现/组网兼容性严重影响演示，再把 Nearby 作为替代 Transport，而不是重写整个业务逻辑。

Bluetooth Classic 可以通过 RFCOMM `BluetoothSocket` 提供字节流，但连接、配对体验以及吞吐使它不适合作为你们视频、GLB 等大媒体的主要传输层；Android 文档还要求阻塞连接操作避免放在 UI 线程。 [7]

BLE 更不适合作为大媒体主链路。Android 官方明确把 BLE 定位为低功耗、传输较少数据的场景，因此它最多适合发现、配对或控制消息，而不适合把几十 MB 视频和模型作为主要 Payload。 [8]

所以：

BLE ≠ 你们的文件传输方案。

OCR：Android 本地和服务器双路线非常合理

Google ML Kit Text Recognition V2 目前提供：

- Latin；
- Chinese；
- Japanese；
- Korean；
- Devanagari；

等脚本文字识别模块。更重要的是，它有 **bundled 和 unbundled 两种安装方式**：bundled 模型直接静态打进 App，首次离线使用不需要再等下载；官方当前文档给出的代价约是每种脚本架构增加约 4 MB，而 Play Services 动态版本约增加 260 KB，但可能需要等待模型首次下载。 [9]

所以对你们这个项目，我会明确建议：

> **Android MVP 直接 bundled Chinese + Latin。**

理由不是为了炫技，而是：

```
用户坐高铁
无互联网
↓
拍票根
↓
照片已经保存
↓
本机 OCR 尝试识别
↓
成功：自动填字段
失败：用户手动补
↓
收藏依然创建成功
```

这就是非常自然的 Offline-first 体验。

服务端则可以增加 **PaddleOCR**。

截至本轮调研，其官方仓库为 Apache 2.0，支持 100+ 语言；当前 PP-OCRv6 还提供统一的多语言模型，可以同时处理中文、英文、日文以及大量拉丁文字语言，并提供 tiny、small、medium 等面向 edge/mobile/server 的不同模型层级。 [10]

因此建议 OCR 架构：

```
                图片
                 |
          Android 本地保存
                 |
        ML Kit 本地 OCR
```

```
          /              \
       成功              一般
        |                 |
   快速填字段          允许人工修改
        |                 |
        └─────────┬───────┘
                  |
                联网
                  |
          PaddleOCR / VLM
                  |
              更强的解析
```

但需要特别理解：

> **OCR ≠ 理解票根。**

OCR 解决的是：

```
图片 → “澳门科技大学 10月12日 18:30 ...”
```

业务解析解决的是：

```
{
 "category": "movie",
 "title": "...",
 "venue": "...",
 "date": "...",
 "seat": "..."
}
```

所以 OCR 后面还需要：

> 文本规则 + 类别模板 + 候选搜索 + 可选 LLM/VLM + 用户确认。

这才是完整的识别管线。

AnyImage：保留，而且应该重新给它正确定位

我同意你之前强调的：

> **AnyImage 完全可以放服务器，甚至放你自己的电脑。**

但经过重新检查，我认为还应该把它从“快速 3D 建模”这个模糊概念里拆出来。

AnyImage 官方文档提供 Plane、Depth Plane、Cutout 等基于图像构造空间效果的能力，也提供 Bake Mesh，将程序化图像对象固化为普通 mesh 和纹理。换言之，它非常适合做**2.5D / 空间化照片 / 立体贴纸式纪念品**，但不应与 Meshy、Hunyuan3D 那种生成完整三维物体的模型混为一谈。 [11]

这反而非常适合你们。

电影票根没必要变成：

> “AI 猜出来一整张不知道背面长什么样的 3D 物体。”

它更适合：

```
票面
┌──────────────────────────────┐
│ 电影海报                     │
│        ↗ 人物层              │
│     ↗ Logo层                 │
│   ↗ 票面层                   │
└──────────────────────────────┘
        ↑
  有深度、有层次
```

这才符合收藏卡的审美。

AnyImage 同组织还有一个非常值得你们关注的 **BlendJob**。官方将它定义为 Blender extensions 的本地 Job Server runtime，可以让长耗时任务运行在独立 Python 环境，并提供进度、取消、FIFO 调度、结果交付和服务器生命周期管理；它本身就面向 AI inference、媒体处理、几何计算等 Blender 主线程之外的任务。 [12]

这意味着：

> **你们不一定需要自己从零解决 Blender 内“怎么安全跑后台任务”这件事。**

我反而建议把你的“打隧道到自己电脑”升级成下面这种正式架构：

```
云服务器
 Job Queue
     │
     │ 有任务吗?
     │
     ▼
你的电脑上的 Worker Agent
     │
     ├── 下载 COS 原图
     │
     ├── 调 Blender / AnyImage
     │
     ├── Bake Mesh
```

```
    │
    └── 上传 GLB / 纹理到 COS
```

关键区别在于：

```
原方案：
Internet → Tunnel → 你的电脑开放服务
```

变成：

```
你的电脑 → 主动向服务器拉任务
```

后一种更符合长期架构，因为个人电脑只需要主动建立 outbound 连接，不要求给它一个稳定公网入口。

你之前提出的：

> “服务器上一秒内响应。”

我认为应该保留为**待验证性能目标**，而不是写进开题报告当事实。

因为“一秒”可能分别意味着：

1. API 1 秒返回 `job_id`；
2. 1 秒完成 AnyImage 运算；
3. 1 秒生成 GLB；
4. 1 秒上传 COS；
5. 1 秒手机已经显示。

这是完全不同的指标。

课程里真正漂亮的做法是以后实测：

> **T_capture → T_queue → T_worker_start → T_generate → T_upload → T_client_ready**

Meshy、高质量开源 3D 与备用路线

Meshy 仍然非常适合作为你们的**高质量模式**。

当前 Image-to-3D API 采用异步任务模式，支持纹理/PBR以及 GLB、OBJ、FBX 等多种结果格式；API 会返回任务状态、进度和模型 URL，而且结果 URL 带有效期信息，所以不能把 Meshy 返回的 URL 当永久数据库字段，应该把最终资产复制进你们自己的 COS。 [13]

Meshy 当前按 credits 计费，例如不同模型、纹理分辨率和几何等级会消耗不同 credits，因此工程上需要任务去重、用户配额和失败重试策略，不能因为客户端重复点击就提交多个付费任务。 [14]

另一个很容易被误解的地方是“动态 3D”。

Meshy 的 Rigging API 并不是通用的“任何东西一键动画”；官方说明其自动绑定重点适用于标准双足人形模型。你们的奶茶杯、食物、票根显然不应该依赖这条路线。 15

你们需要的动态大多可以直接由 Three.js 做：

```
旋转
悬浮
翻面
卡片展开
光照变化
视差
点击爆开
前后层分离
```

并不需要真正骨骼动画。

同时我找到了几条非常值得留在技术储备中的开源路线。

方案	本质	官方给出的资源/速度线索	本项目建议
AnyImage	2.5D / 图像空间化	Blender 工作流	即时视觉模式
TripoSR	单图 → 3D	默认约 6 GB VRAM；官方报告 A100 下可低于 0.5 秒	快速开源备用
Stable Fast 3D	单图 → Mesh + UV + Texture	默认约 6 GB VRAM，可输出 GLB	很值得实验
Hunyuan3D 2.x	Shape + Texture	官方称 shape 约 6 GB；shape+texture 合计约 16 GB VRAM	高质量自托管实验
Meshy	云端商业 API	无需自管 GPU	MVP 高质量模式

TripoSR 官方仓库是 MIT License，并声称在 NVIDIA A100 上单图生成可低于 0.5 秒，默认单图配置大约需要 6 GB VRAM；同时提供 texture baking。注意，这个官方数字不能外推成“你的服务器也一定 0.5 秒”。 16

Stable Fast 3D 在 TripoSR 基础上强调更好的 mesh、UV unwrapping、纹理以及材质参数，默认单图路线同样约 6 GB VRAM，并直接输出 GLB；它还提供 CPU 和实验性 Apple MPS 路线。 17

腾讯 Hunyuan3D 2.x 则采用形状生成和纹理生成两个阶段；当前官方仓库给出的资源需求约为 shape 6 GB VRAM、shape + texture 总计 16 GB，同时 Hunyuan3D-2.1 已开放新的 PBR 模型、VAE encoder 和训练代码。 18

所以我会给你们定成：

```
Level A: 2D 卡片
       ↓ 几乎立即
Level B: AnyImage 2.5D / 立体卡
```

```
        ↓ 快速
Level C: Meshy 高质量 3D
        ↓ 可等待
Level D: Hunyuan / SF3D 自托管实验
          加分，不阻塞产品
```

这比“所有票根都必须先生成完整 3D 才能收藏”合理太多。

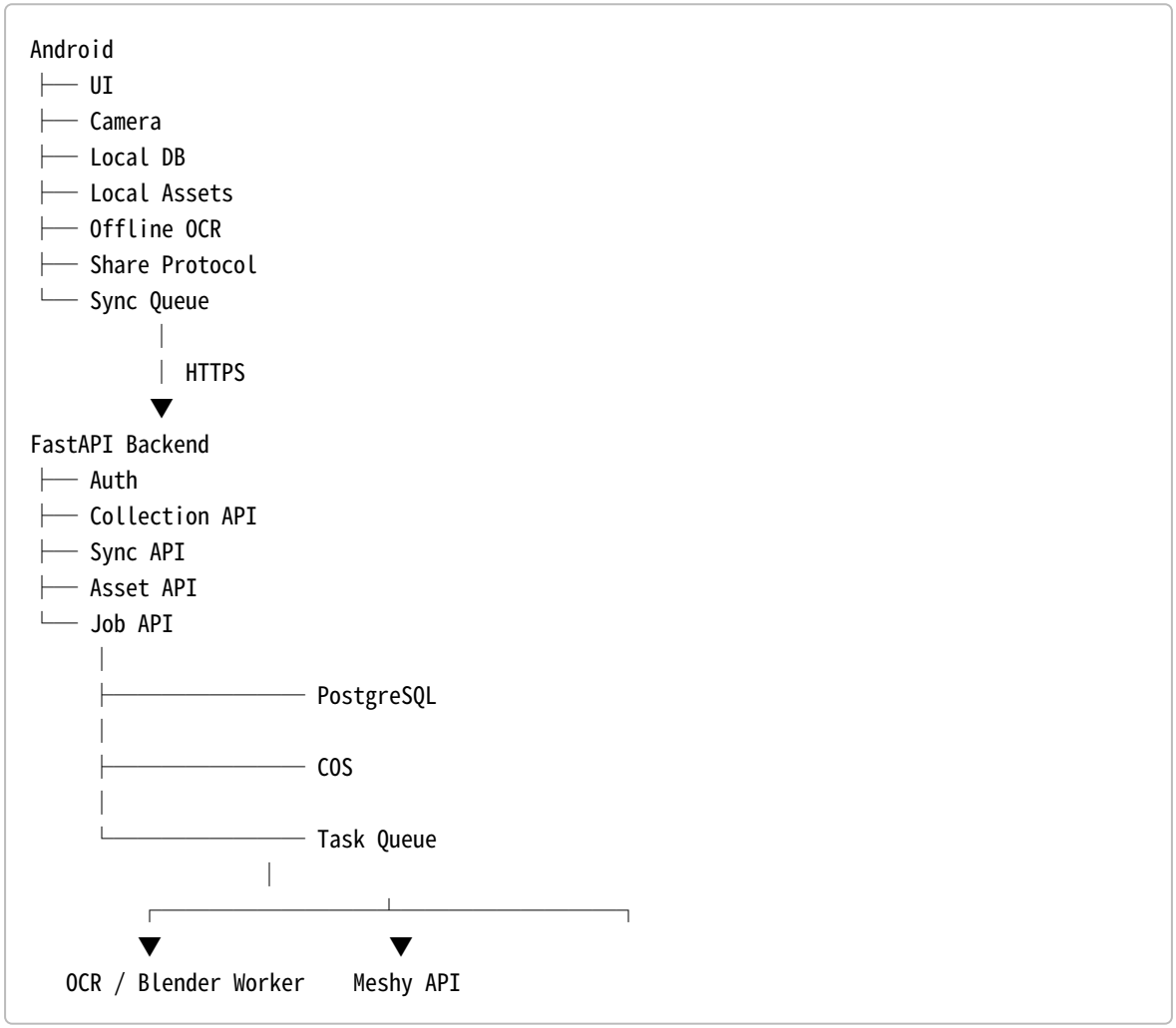
另外，AnyImage 仓库使用 GPL-3.0-or-later；若你们修改并分发该插件或把相关代码作为项目交付的一部分，需要专门检查许可证义务。把它作为服务器 Worker 使用可以降低与 Android/Web 客户端代码直接耦合的问题，但这不应被视为法律意见。 [19]

推荐架构与真正的离线能力

我现在推荐的总架构如下：

电子票根项目总体架构

最重要的思想不是“微服务”，而是**职责清楚**。



Capacitor 在这里非常合适

Capacitor 并不是“把网页网址套壳”，而是允许 Web 技术构建的应用运行在原生容器中，并通过插件调用平台原生 API；官方文档也提供自定义插件机制，让 TypeScript API 与 Android/iOS 原生实现连接。 [20]

因此你们可以：

```
React + TypeScript
 ├── 首页
 ├── 收藏册
 ├── 卡片
 ├── Three.js
 ├── 搜索
 └── 详情页
```

基本共享给 Web 和 Android。

只有下面这些东西写 Kotlin：

```
NativeBridge
 ├── Wi-Fi Direct
 ├── TCP Socket
 ├── Nearby（若需要）
 ├── ML Kit
 ├── Android 文件能力
 └── 特定后台行为
```

本地数据库可以直接采用 `@capacitor-community/sqlite`。该项目当前支持 Android、iOS、Electron 和 Web；在 Native 端使用 SQLite/SQLCipher，在 Web 端通过 IndexedDB 等机制支撑其 Web 实现，并采用 MIT License。 [21]

这意味着甚至可以做：

```
interface LocalRepository {
  saveCollectible(...)
  listCollectibles(...)
  addSyncOperation(...)
}
```

Android 背后是 SQLite。

Web 背后可以是 IndexedDB。

这样 UI 不需要知道底层到底是什么。

COS 应该成为一等公民

你之前问：

> COS、OSS、S3 是不是对象存储?

是。

你们只需要选一个，比如腾讯云 COS。

正确分工应该是：

存储	保存什么
PostgreSQL	用户、收藏 ID、标题、时间、分类、权限、版本、任务状态
COS	JPG、PNG、MP4、GLB、纹理、海报、缩略图
Android SQLite	离线元数据、缓存索引、sync queue
Android Files	已下载原图、视频、模型、缩略图

最典型的错误架构是：

```
PostgreSQL
 └──── 直接存 100 MB 视频
```

或者：

```
Android App
 └──── 内置腾讯云永久 SecretKey
```

后者尤其不能做。

腾讯云 COS 官方当前明确把 temporary keys 定位于前端直接上传等临时授权场景，并建议遵循最小权限原则；临时凭证可以限制到特定 action、bucket 和 path，避免把全 bucket 权限交给客户端。 22

正确方式应该是：

```
Android
  │
  │ 我要上传 assets/123
  ▼
Backend
  │
  │ 校验登录用户
  │
  │ 请求 STS
```



这样 100 MB 视频不必先绕业务服务器一遍。

“离线”应该拆成三个等级

你们以后不要在开题报告中写一句：

> “系统支持离线。”

这太模糊。

应该定义：

A0：Local Offline

没有服务器，没有互联网。

可以：

> 拍照、导入、浏览已有收藏、编辑、查看已下载媒体、ML Kit OCR、创建收藏。

ML Kit bundled 模型可以直接随 Android App 分发，不需要首次在线下载，这是它对本项目最大的价值。 9

A1：Nearby Offline

没有互联网，但两台 Android 的 Wi-Fi / Bluetooth 无线能力可用。

在 A0 基础上增加：

> 设备发现、连接、收藏包分享、媒体传输、导入。

Wi-Fi Direct 本身就支持在没有传统接入点的情况下建立 P2P 连接；Nearby Connections 同样明确面向无互联网附近设备的数据交换。 23

A2：Online

增加：

> 云同步、COS、电影元数据、PaddleOCR、Meshy、远程 AnyImage Worker、Web 跨设备访问。

这个划分会让你们的产品说明一下子严谨很多。

Web 端建议不承担“强离线”验收。

Web 可以以后做 PWA 缓存和 IndexedDB，但一个月范围内：

> Android = Local-first
> Web = Cloud companion

已经足够漂亮。

同步不要上来就做 CRDT

这个项目没必要为了高级而高级。

你们多数数据属于：

> 一个用户编辑自己的私人收藏。

所以一个很实用的同步协议是：

```
本地修改
   ↓
SQLite transaction
   ↓
写 sync_operation
   ↓
op_id
entity_id
base_revision
operation
payload
   ↓
恢复联网
   ↓
POST /sync/push
   ↓
服务器按 op_id 去重
   ↓
更新 revision
   ↓
GET /sync/pull?cursor=...
```

删除使用 tombstone：

```
{
 "id": "...",
 "deleted": true,
```

```
  "revision": 18
}
```

避免旧手机再次同步时把已经删除的收藏“复活”。

PowerSync 确实是一个可以备用的现成方案：官方将其定义为让客户端 SQLite 与后端数据库同步、支持实时与离线工作的同步引擎；同时目前提供 Cloud、免费的 source-available Open Edition 以及企业自托管版本。[24]

但我反而**不建议一开始用 PowerSync**。

原因不是它不好，而是你这个是网络编程课程。

自己实现一个受控范围的：

> Outbox + Idempotency + Revision + Cursor + Tombstone

比：

> “我们用了同步 SDK”

更容易向老师证明网络和分布式系统理解。

网络编程难度与一点三评分论证

这是整个项目最值得认真设计的一部分。

我重新评估以后认为：

> **项目具备非常充分的 1.30 申报潜力，但不是现在画完架构图就已经等于 1.30。**

真正的分界线在：

你们自己解决了多少“网络问题”。

下面两套实现表面看上去功能一样：

版本 A

```
手机 A
 ↓
sendFile()
 ↓
手机 B
```

版本 B

```
Discovery
   ↓
Peer Selection
   ↓
Connection
   ↓
Version Negotiation
   ↓
Offer
   ↓
Manifest
   ↓
Chunk Transfer
   ↓
Disconnect
   ↓
Reconnect
   ↓
Resume
   ↓
SHA-256 Verify
   ↓
Atomic Import
   ↓
Receipt ACK
```

网络编程老师看到这两个项目，对难度的判断完全不同。

Google Nearby Connections 本身已经抽象了 Bluetooth、BLE、Wi-Fi 的底层传输、连接和 payload 机制，因此它非常适合产品开发，但也意味着直接使用 SDK 会减少你们能够展示的底层网络设计。 4

这也是我为什么建议：

> **Wi-Fi Direct + TCP 做课程技术主线。**

建议你们真正实现一个应用层协议

名字甚至可以正式取：

> **MTP — Memorabilia Transfer Protocol**

或者：

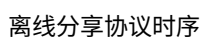
> **MCPack Transfer Protocol**

注意后者不要和 Model Context Protocol 的 MCP 混淆，所以 MTP 更好。

协议流程：

```
DISCOVER
   ↓
CONNECT
   ↓
HELLO
   protocol_version
   capabilities
   device_id
   ↓
OFFER
   title
   category
   preview
   ↓
ACCEPT
   ↓
MANIFEST
   package_id
   snapshot_id
   assets[]
   size
   mime
   sha256
   ↓
NEED / RESUME
   ↓
CHUNK
   ↓
VERIFY
   ↓
IMPORT_COMMIT
   ↓
RECEIPT
```

可视化如下：



离线分享协议时序

这里面非常多都是实打实的网络编程问题：

消息边界。

TCP 是字节流，所以：

```
send(A)
send(B)
```

接收端并不天然对应：

```
recv(A)
recv(B)
```

你们需要自己的 framing，例如：

```
MAGIC
VERSION
TYPE
LENGTH
PAYLOAD
```

大文件分块。

例如：

```
Asset 83 MB
↓
1 MB × 83 chunks
```

断点续传。

接收端已经有：

```
0...37 chunk
```

重新连接后：

```
RESUME asset_123 from chunk 38
```

而不是重新发 83 MB。

完整性校验。

Manifest：

```
{
 "asset_id": "...",
 "size": 87382111,
 "sha256": "..."
}
```

接收后验证。

幂等导入。

同一个：

```
package_id = ABC
```

因为断线重试收到两次，不能在收藏册出现两张完全相同的卡。

事务提交。

不能：

```
数据库已经显示收藏
但是视频只传了 17%
```

却告诉用户“完整导入成功”。

应该有：

```
Receiving
↓
Verified
↓
Imported
↓
Committed
```

这些设计叠加起来，网络编程味道已经非常重了。

不要把 TCP 本身已经解决的问题再假装自己实现

需要特别注意：

TCP 已经负责：

> 有序传输、丢包重传、流量控制等底层可靠机制。

所以课程报告不能写：

> “我们自己实现了 TCP 丢包重传。”

除非你真的在 UDP 上做。

你们自己解决的是：

application-level resume、business transaction、content integrity、session recovery。

这反而更专业。

我对难度等级的实际判断

因为老师没有明确给出可计算公式，下面只能视为**项目内部估计，不是教师最终评分预测**。

最终完成程度	我的难度判断
React + FastAPI + DB + OCR API	1.10 左右
再加 Meshy + COS + Three.js	1.10–1.20
再加 Android 本地数据库 + 离线 OCR	1.20 左右
再加 Wi-Fi Direct + TCP 自定义协议	接近/进入 1.30
再加断点恢复 + 幂等 + 完整性验证	很强的 1.30 依据
再加 Local-first 云同步	进一步加强
再加远程 Blender Worker / 任务系统	非网络复杂度加分
再加双 Transport 或自托管 3D	可作为更高难度展示

所以，我现在会把这个项目定义成：

> **“稳健冲 1.30，而不是冒险堆技术冲 1.40。”**

你不需要为了 1.30 再去实现 NAT hole punching。

你们的业务根本不需要跨公网 P2P。

两个人面对面分享：

```
Wi-Fi Direct
```

比：

```
STUN
TURN
ICE
NAT Traversal
```

更贴合产品需求。

不为了技术而技术，本身也是好的架构判断。

技术价值与交付风险的内部分析如下：

技术价值风险矩阵

其中我认为最值得投入的是：

Wi-Fi Direct + TCP、离线同步、AnyImage Worker。

而最容易变成“技术很多但是没多少产品价值”的，是：

MCP Server、Hunyuan3D 自托管、第二套近场通信 Transport。

所以后者全部应当作为 Stretch，而不是核心路径。

风险、逻辑漏洞与改进方向

目前最大的风险已经不是“代码写不写得完”。

你们四个人一个月，再加很强的 AI 辅助，纯代码生产速度可能不是瓶颈。

真正风险是：

错误的需求在 AI 的帮助下被非常快速地实现。

所以现在最重要的是把业务语义冻结到足够稳定。

最大产品风险：什么都想自动识别

目前输入包括：

电影票、高铁票、奶茶贴纸、食物照片、店铺视频……

但这些输入的信息可获得性完全不同。

电影票：

```
片名 → 可以查电影数据库
```

奶茶：

```
“霸王茶姬”
“伯牙绝弦”
→ OCR 可能识别
```

一张食物照片：

```
一碗面
```

图片本身并不能可靠推出：

> “这是澳门 XXX 店的 XXX 牛肉面。”

所以不能设计成：

> AI 必须自动猜出所有事实。

正确 UX 是：

```
模型非常确定
→ 自动填

模型有候选
→ 让用户点选

模型不确定
→ “待补充”

用户知道
→ 手工填
```

趣味内容可以生成，事实字段不能装作确定。

MVP 应该只把一类“自动知识匹配”做深

我最建议：

> **电影。**

原因不是电影最酷，而是产品闭环最完整。

```
拍电影票
↓
OCR
↓
片名
↓
搜索
↓
选电影
↓
海报 / 年份 / 导演 / 简介
```

```
↓
票根数字卡
```

TMDB 的官方 API 就是用于让应用访问电影、电视剧、人物数据与图像；对于非商业使用目前允许免费使用，但明确要求 attribution，包括在应用中加入规定的 TMDB 声明。 [25]

所以开题报告里可以写：

> 电影类提供深度自动匹配。

而不是写：

> 系统自动匹配所有票根对应的网络资料。

高铁：

> OCR + 字段抽取 + 用户确认。

奶茶：

> OCR + 图片 + 用户确认。

探店：

> 视频 + 用户自己填店名或在线候选。

这样一个月的实际质量会高很多。

分享时不要把整张原始票据默认发出去

这是一个我认为应该直接定成产品原则的事情。

分享包不应该默认：

```
整个数据库对象
+
原始OCR结果
+
原票照片
+
所有视频
+
所有模型
```

而应该是：

```
Share Snapshot
├── 名称
├── 类型
├── 推荐语
├── 海报/封面
├── 用户允许分享的时间/地点
├── 可选视频
└── 可选3D
```

原票照片单独开关。

尤其原始票据可能出现：

> 姓名、订单号、座位、二维码、行程信息等。

分享采用字段白名单，比字段黑名单安全得多。

例如：

```
{
 "share": {
  "title": true,
  "poster": true,
  "recommendation": true,
  "venue": true,
  "date": false,
  "original_ticket": false
 }
}
```

这是一个很容易在答辩中体现工程成熟度的细节。

大型素材不能阻塞“立刻看到”

你最初的想法是：

> A 分享给 B，B 立刻看到这个电影叫什么。

那协议就不应该先发送：

```
70 MB GLB
↓
200 MB 视频
↓
最后发送 title
```

正确顺序：

```
1 KB 元数据
↓
100 KB thumbnail
↓
B 已经可以看
↓
海报
↓
视频
↓
3D
```

也就是：

Time to First Meaningful Content

而不是只优化：

Total Transfer Time。

这实际上非常适合作为你们网络实验的一个指标。

AnyImage 不应该通过“公网隧道到个人电脑”成为单点依赖

你说“可以打隧道到我电脑”是可行的开发方式，但正式架构最好不要绑定它。

建议：

```
Worker 1: 云 GPU
Worker 2: 你的 PC
Worker 3: 组员 PC
```

它们统一：

```
register worker
↓
pull job
↓
heartbeat
↓
claim
↓
download asset
↓
process
↓
upload
```

```
↓
complete
```

这样项目会突然变成一个很有意思的**分布式计算节点系统**。

BlendJob 已经提供本地 Blender Job Server 的长任务、进度、取消和队列机制，可以利用它做每一个 Worker 内部的任务运行时，而你们自己负责“云调度器 ↔ Worker”的网络层。 12

这比：

> “服务器 ssh 到我电脑运行 Blender。”

漂亮很多。

3D 不应该成为收藏创建的强依赖

必须允许：

```
收藏 CREATED
3D PENDING
```

而不是：

```
3D FAILED
↓
收藏 FAILED
```

建议资产状态：

```
collectible
├── source_image READY
├── poster READY
├── instant_3d READY
├── premium_3d PROCESSING
└── video LOCAL_ONLY
```

这样后端任务完全可以异步。

Meshy 结果必须归档

因为 Meshy API 的模型结果 URL 存在有效期，因此长期收藏系统应该把需要保留的 GLB、贴图等结果复制进自己的对象存储，而不是数据库永久保存 Meshy URL。 13

因此：

```
Meshy
  ↓
Temporary result URL
  ↓
Worker downloads
  ↓
COS
  ↓
asset.object_key
```

而不是：

```
database:
model_url = "https://meshy......Expires=..."
```

COS 权限不能图省事

腾讯云明确建议临时凭证按业务需求实施最小权限，并可以限制 action 和具体资源范围。[22]

所以不要：

```
resource: *
action: *
```

给 App。

而应该类似：

```
user A
↓
只可以
users/A/uploads/{random-id}
↓
PutObject
```

课程报告把这一点写进去，鉴权/认证的技术价值也会明显提高。

海报不是“API 能取就随便用”

TMDB 明确要求 attribution；其 FAQ 也区分开发者非商业使用与商业使用，并对 API 和图像来源声明有要求。[25]

所以课程 Demo 没问题，但产品设计里应该有：

`asset_source = TMDB`

以及 Credits 页面。

这是工程边界，不必把它发展成法律研究项目，但不能完全忽略。

最终建议的 MVP、验收标准与下一轮决策

经过这一轮研究，我会把你们的最终范围从“大而全的各种 AI 功能”收束成三个层级。

Core：没有它就不是这个产品

能力	要求
Android 拍照 / 图片 / 视频导入	必须
本地收藏与浏览	必须
手工编辑结构化信息	必须
Web 收藏查看	必须
FastAPI + PostgreSQL	必须
COS 对象存储	必须
用户鉴权	必须
Android 无互联网两机分享	必须
分享语义正确	必须

Target：这部分决定项目是否真正漂亮、是否足以论证 1.30

能力	推荐
Wi-Fi Direct + TCP	核心网络技术
MTP 自定义协议	核心
分块 + Resume	核心
SHA-256 完整性	核心
幂等导入	核心
ML Kit 离线 OCR	强烈建议
PaddleOCR 在线增强	建议
Outbox 云同步	强烈建议
AnyImage Worker	建议
Meshy API	建议
Three.js 视觉效果	建议

Stretch：有时间就非常加分，没时间不伤产品

能力	判断
Nearby Connections 第二 Transport	兼容性增强
Hunyuan3D 自托管	技术展示
Stable Fast 3D	技术实验
MCP Server	技术扩展
Web 完整离线	非核心
iOS	本期不做
NAT 穿透	没有业务必要
公共社交网络	Scope 太大
区块链/NFT	与产品价值无关

最后，我建议你们以后向老师演示的不是：

> “这是我们首页，这是 OCR，这是 Three.js，这是 Meshy。”

而是一整段故事：

> **手机 A 在没有互联网时拍下一张电影票。**
>
> 本地保存立即成功。
>
> ML Kit 离线识别片名与日期。
>
> A 修改推荐语：“这部电影很好看。”
>
> A 点击“递给附近朋友”。
>
> 手机 B 被发现。
>
> 双方建立 Wi-Fi Direct/TCP 连接。
>
> MTP 首先发送电影名、卡片和预览。
>
> B 几乎立刻看到内容。
>
> 开始发送视频/3D。
>
> 中途人为关掉连接。
>
> 再次连接。
>
> 从中断位置继续。
>
> SHA-256 验证成功。

B 原子导入收藏。

B 的状态是“朋友推荐 / 想看”，而不是“已看”。

两台手机重启，收藏仍在。

恢复互联网。

Outbox 自动同步。

打开电脑 Web。

B 刚收到的收藏已经出现。

同时后台 AnyImage / Meshy 任务完成。

WebSocket/SSE 或轮询收到任务完成状态。

COS 中的 GLB 被加载。

Three.js 把普通卡片切换成具有立体效果的数字纪念品。

如果这整条链能够稳定演示，项目呈现出来的就不再是：

> **“一个调用很多 API 的学生 App”。**

而会是：

> **“一个具备 Local-first 数据模型、P2P 近场网络协议、断线恢复、云同步、对象存储、异步计算节点、多模态识别与三维内容流水线的完整网络应用。”**

这就是我认为最有说服力的 **1.30 难度论证**。

同时，下一轮最值得逐条冻结的不是更多技术，而是以下几个产品决策：**产品最终叫“电子票根”还是“生活纪念品”；电影是否作为唯一深度自动匹配类别；Wi-Fi Direct + TCP 是否正式成为课程网络主线；分享是否默认排除原票照片与敏感字段；AnyImage 是否正式定义为“即时 2.5D”，Meshy 定义为“高质量 3D”；Web 是否不承担严格离线；同步是否采用不可变 Snapshot + Outbox/Revision，而不引入 CRDT；MCP 是否只作为最后的展示性扩展。**这些决定一旦明确，后续 PRD、数据库 Schema、API、MTP 协议和四人并行开发就可以迅速稳定下来。

---

[1] [3] PocketStubs: Movie TV Tracker, - Apps on Google Play
https://play.google.com/store/apps/details?hl=en&id=com.pocketstubs.app

[2] TicketShelf: Concert Stub Log - Apps on Google Play
https://play.google.com/store/apps/details?hl=en&id=com.defineyou.ticketshelf

[4] [23] Overview | Nearby Connections | Google for Developers
https://developers.google.com/nearby/connections/overview

5 Strategies | Nearby Connections | Google for Developers
https://developers.google.com/nearby/connections/strategies

6 Wi-Fi Direct (peer-to-peer or P2P) overview | Connectivity | Android Developers
https://developer.android.com/develop/connectivity/wifi/wifip2p

7 Connect Bluetooth devices | Connectivity | Android Developers
https://developer.android.com/develop/connectivity/bluetooth/connect-bluetooth-devices

8 Bluetooth Low Energy | Connectivity | Android Developers
https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview

9 Recognize text in images with ML Kit on Android | Google for Developers
https://developers.google.com/ml-kit/vision/text-recognition/v2/android

10 GitHub - PaddlePaddle/PaddleOCR: Turn any PDF or image document into structured data for your AI. A powerful, lightweight OCR toolkit that bridges the gap between images/PDFs and LLMs. Supports 100+ languages. · GitHub
https://github.com/PaddlePaddle/PaddleOCR

11 raw.githubusercontent.com
https://raw.githubusercontent.com/OmooLab/AnyImage/main/docs/index.md

12 GitHub - OmooLab/BlendJob: Reusable local Job Server runtime for Blender add-ons · GitHub
https://github.com/OmooLab/BlendJob

13 Image to 3D API | Meshy Docs
https://docs.meshy.ai/en/api/image-to-3d

14 Pricing | Meshy Docs
https://docs.meshy.ai/en/api/pricing

15 Rigging API | Meshy Docs
https://docs.meshy.ai/en/api/rigging

16 GitHub - VAST-AI-Research/TripoSR: TripoSR: Fast 3D Object Reconstruction from a Single Image · GitHub
https://github.com/VAST-AI-Research/TripoSR

17 GitHub - Stability-AI/stable-fast-3d: SF3D: Stable Fast 3D Mesh Reconstruction with UV-unwrapping and Illumination Disentanglement · GitHub
https://github.com/Stability-AI/stable-fast-3d

18 GitHub - Tencent-Hunyuan/Hunyuan3D-2: High-Resolution 3D Assets Generation with Large Scale Hunyuan3D Diffusion Models. · GitHub
https://github.com/Tencent-Hunyuan/Hunyuan3D-2

19 raw.githubusercontent.com
https://raw.githubusercontent.com/OmooLab/AnyImage/main/LICENSE

20 Creating Capacitor Plugins | Capacitor Documentation
https://capacitorjs.com/docs/plugins/creating-plugins

21 GitHub - capacitor-community/sqlite: ⚡Capacitor plugin for native & electron SQLite databases. · GitHub
https://github.com/capacitor-community/sqlite

22 Generating and Using Temporary Keys
https://www.tencentcloud.com/document/product/436/14048

24 PowerSync Docs - PowerSync
https://docs.powersync.com/intro/powersync-overview

25 FAQ
https://developer.themoviedb.org/docs/faq